

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ITMO University

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ 7-8

По дисциплине Web-программирование

Тема работы Разработка проекта на Django

Обучающийся Алексеев Тимофей Юрьевич

Факультет Факультет инфокоммуникационных технологий

Группа K3321

Направление подготовки 11.03.02 Инфокоммуникационные технологии и системы связи

Образовательная программа Программирование в инфокоммуникационных системах

Обучающийся	<u>05.05.2025</u> (дата)	<u> </u> (подпись)	<u>Алексеев Т.Ю.</u> (Ф.И.О.)
Руководитель	<u> </u> (дата)	<u> </u> (подпись)	<u>Марченко Е.В.</u> (Ф.И.О.)

Санкт-Петербург
2025 г.

Цель

Создать проект на Django с нуля, включающий три страницы, форму и футер с хэдером и используя возможности фреймворка.

Задачи

1. Установка Django;
2. Реализация основной структуры проектов, футера и хэдера;
3. Реализация страницы «Главная»;
4. Реализация страницы обратной связи с формой;
5. Реализация страницы «О разработке».

Ход работы

Для начала работы необходимо установить Django. Сделаем это в отдельном проекте с помощью утилиты `pip`.

Будем создавать сайт для заказа логотипа, который будет содержать в себе 3 страницы: Главную, страницу формы для заказа логотипа и о разработке.

Для создания пустой структуры проекта пропишем следующие команды:

```
django-admin startproject logo_site
```

```
cd logo_site
```

```
python manage.py startapp orders
```

Далее сразу перейдем к созданию модели для формы заказа логотипа пользователем.

В `models.py` создадим класс, отвечающий за ответы на форму пользователей, в которой создадим поля, содержащие ответы пользователя и время заполнения формы.

```
class LogoOrder(models.Model):
    full_name = models.CharField("Фамилия Имя", max_length=200)
    contact = models.CharField("Контактный телефон или Телеграмм", max_length=200)
    description = models.TextField("Какой логотип желаете заказать?")
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"Заказ от {self.full_name} ({self.created_at.strftime('%Y-%m-%d %H:%M')})"
```

Рисунок 1 – Код `models.py`

После этого необходимо написать класс формы, который будет использоваться в `html`-файле страницы. Напишем его в `forms.py`, где укажем надписи и имена полей.

```
class LogoOrderForm(forms.ModelForm):
    class Meta:
        model = LogoOrder
        fields = ['full_name', 'contact', 'description']
        labels = {
            'full_name': 'Фамилия Имя',
            'contact': 'Контактный телефон или Телеграм',
            'description': 'Какие пожелания для логотипа?',
        }
        widgets = {
            'description': forms.Textarea(attrs={'rows':4}),
        }
```

Рисунок 2 – Код forms.py

После того, как форма будет готова, необходимо написать один из самых важных файлов – файл представления views.py.

В нем указаны все функции, вызывающие соответствующие html-файлы и передающие в них требуемые аргументы. В каждую из них будем передавать нынешний год (необходим для отрисовки футера). Также, в страницу «О разработке» будем передавать количество имеющихся записей в базе данных. Также, при заполнении формы она будет сохраняться и будет возвращаться страница с успешным заполнением формы.

```

def home(request):
    return render(request, 'orders/home.html', {'current_year': datetime.now().year})

def about(request):
    logo_order_count = LogoOrder.objects.count()

    context = {
        'logo_order_count': logo_order_count,
        'current_year': datetime.now().year,
    }
    return render(request, 'orders/about.html', context)

def order_logo(request):
    if request.method == 'POST':
        form = LogoOrderForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect('order_success')
    else:
        form = LogoOrderForm()

    return render(request, 'orders/order_logo.html', {'form': form, 'current_year': datetime.now().year})

def order_success(request):
    return render(request, 'orders/order_success.html', {'current_year': datetime.now().year})

```

Рисунок 3 – Код views.py

После создания файла-представлений необходимо написать файлы, отвечающие за url-адреса. Таких файла два: в папке website и в папке orders.

В папке в директории website напомним паттерны url-адресов для разграничения административной панели и общедоступной части.

```

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('orders.urls')),
]

```

Рисунок 4 – Код website/urls.py

В папке в директории orders заполним паттерны url-адресов для работы с адресом и вызова соответствующих функций-представлений.

```
urlpatterns = [
    path('', views.home, name='home'),
    path('order-logo/', views.order_logo, name='order_logo'),
    path('about/', views.about, name='about'),
    path('order-success/', views.order_success, name='order_success'),
]
```

Рисунок 5 – Код orders/urls.py

Также, необходимо прописать код для файла admin.py, который отвечает за работу административной части и запись отправленных пользователем данных.

```
@admin.register(LogoOrder)
class LogoOrderAdmin(admin.ModelAdmin):
    list_display = ('full_name', 'created_at')
    search_fields = ('full_name', 'description')
    ordering = ('-created_at',)
```

Рисунок 6 – Код для admin.py

Кроме того, необходимо создать суперпользователя (администратор) с помощью команды *python manage.py createsuperuser*.

Далее требуется написать html-файлы для каждой страницы сайта. В первую очередь, создадим в папке orders/templates файл base.html, содержащий в себе логику отрисовки футеров и хэдеров. Также, в этом файле подгружаются статические элементы, а именно style.css, использующийся в разметке.

В хэдере будут навигационные элементы для перехода между страницами, а в футере будет кнопка для возвращения на Главную и правила копирайтинга, использующиеся на многих сайтах.

```

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>{% block title %}Сайт для заказа логотипа{% endblock %}</title>
  {% load static %}
  <link rel="stylesheet" href="{% static 'orders/css/styles.css' %}">
</head>
<body>

  <header>
    <h1>Сайт для заказа логотипа</h1>
    <nav>
      <ul class="nav-menu">
        <li><a href="{% url 'home' %}">Главная</a></li>
        <li><a href="{% url 'order_logo' %}">Заказать логотип</a></li>
        <li><a href="{% url 'about' %}">О разработке</a></li>
      </ul>
    </nav>
  </header>

  <main>
    {% block content %}
    {% endblock %}
  </main>

  <footer style="margin-top: 2em; border-top: 1px solid #ccc; padding-top: 1em; text-align: center;">
    <p style="margin-bottom: 3rem;"><a class="footer-back" href="{% url 'home' %}">Вернуться на главную</a></p>
    <p>© {{ current_year }} Сайт для дисциплины WEB-программирование. Все права защищены.</p>
  </footer>

</body>

```

Рисунок 7 – Код base.html

Все последующие html-файлы будут наследовать написанный выше файл. Далее напишем файл для Главной страницы в файле home.html. В нем в качестве статического элемента будет загружаться картинка, отображающаяся на странице. Более того, под картинкой будет кнопка, отправляющая пользователя на страницу с формой.

```

{% extends 'orders/base.html' %}
{% load static %}
{% block title %}Главная{% endblock %}

{% block content %}
<div style="text-align:center">
  <h2>Добро пожаловать в агенство заказа логотипов!</h2>
</div>

<p>Если вашему бизнесу или проекту необходим логотип, то наша компания готова создать вам таковой! Вы только взгляните,
в какой мастерской мы работаем!</p>

<div style="display: flex; justify-content: center;">
  
</div>

<p>Наши художники нарисуют вам логотип под любые ваши требования! Необходимо лишь заполнить форму, нажав кнопку ниже!</p>

<div style="display: flex; justify-content: center;">
  <button onclick="location.href='{% url 'order_logo' %}'" class="button-great">Перейти к заказу логотипа</button>
</div>
{% endblock %}

```

Рисунок 8 – Код home.html

Напишем файл `order_logo.html`, который представляет собой страницу с формой. Здесь используется форма, написанная ранее.

```
{% extends 'orders/base.html' %}
|
{% block title %}Заказать логотип{% endblock %}

{% block content %}
<div style="text-align:center">
  <h2>Форма заказа логотипа</h2>
</div>
<br>
<form method="post" class="order-form">
  {% csrf_token %}
  {{ form.as_p }}
  <div style="display: flex; justify-content: center">
    <button type="submit">Отправить заказ</button>
  </div>
</form>

{% if form.errors %}
<div style="...">
  Пожалуйста, исправьте ошибки в форме.
</div>
{% endif %}
{% endblock %}
```

Рисунок 9 – Код `order_logo.html`

После этого напишем `order_success.html` – страница, сообщающая об успешном заполнении формы.

```
{% extends 'orders/base.html' %}
|
{% block title %}Заказ принят{% endblock %}

{% block content %}
<h2>Спасибо за ваш заказ!</h2>
<p>Ваш заказ на логотип успешно принят и будет обработан в ближайшее время.</p>
{% endblock %}
```

Рисунок 10 – Код `order_success.html`

По аналогии с файлом `home.html` напишем файл `about.html`. В нем также используется картинка из папки `static`. Более того, на этой странице отображается количество записей, содержащееся в базе данных.

```
{% extends 'orders/base.html' %}
{% load static %}
{% block title %}0 разработке{% endblock %}

{% block content %}
<div style="text-align:center">
  <h2>0 разработке</h2>
</div>

<p>В этом разделе вы можете узнать подробнее про процесс создания логотипов и создание нашего сайта.</p>

<p>Все логотипы создаются вручную, каждый художник будет отвечать за то, каким получится логотип.
  Вы будете с ними на связи в течение всего процесса. И сможете контролировать каждую мелочь.</p>

<div style="display: flex; justify-content: center;">
  
</div>

<p>Касаемо разработки нашего сайта, он разработан с помощью Django. Наш сайт имеет футер и хэдер,
  также статические элементы, вроде картинок и css стилей.</p>

<p>Также, наш сайт работает с SQLite, в которую записываются все данные.
  <span style="font-weight: bold">Например, в данный момент у магазина {{ logo_order_count }} заказа(ов).</span></p>
{% endblock %}
```

Рисунок 11 – Код `about.html`

После этого необходимо в файле `settings.py` в `INSTALLED_APPS` добавим «orders».

Далее запустим наш сервер с помощью команды `python manage.py runserver`.

После перехода на `localhost` мы видим главную страницу.

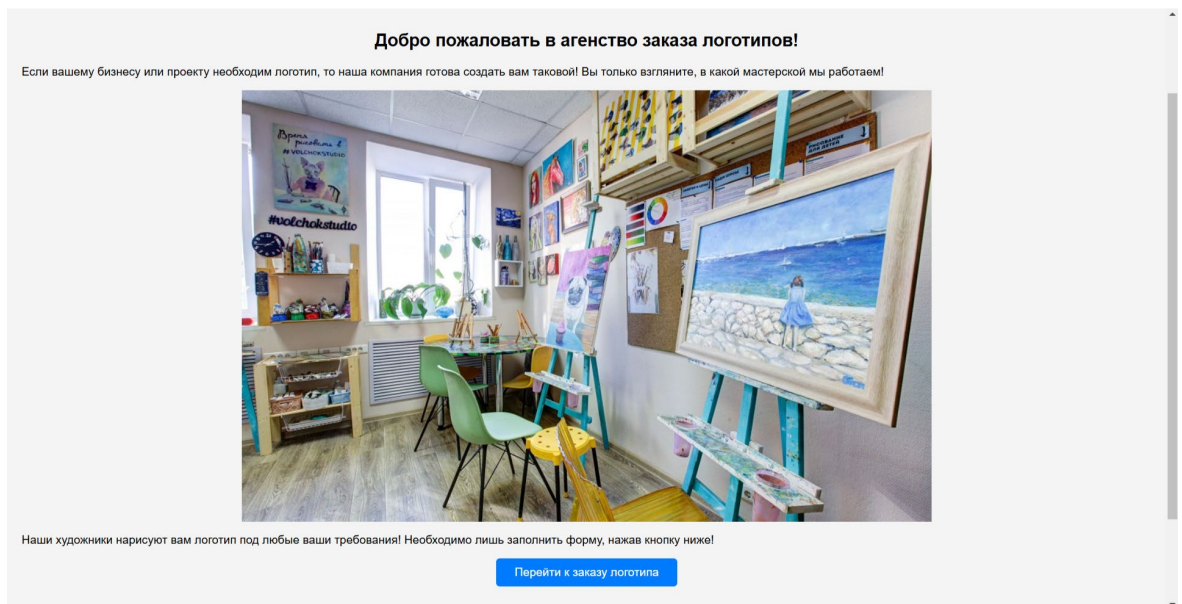


Рисунок 12 – Страница «Главная»

При нажатии на соответствующую кнопку открывается форма для заказа логотипа.

Рисунок 13 – Страница с «Заказ логотипа»

После заполнения формы пользователь увидит следующую страницу об успехе заполнения.

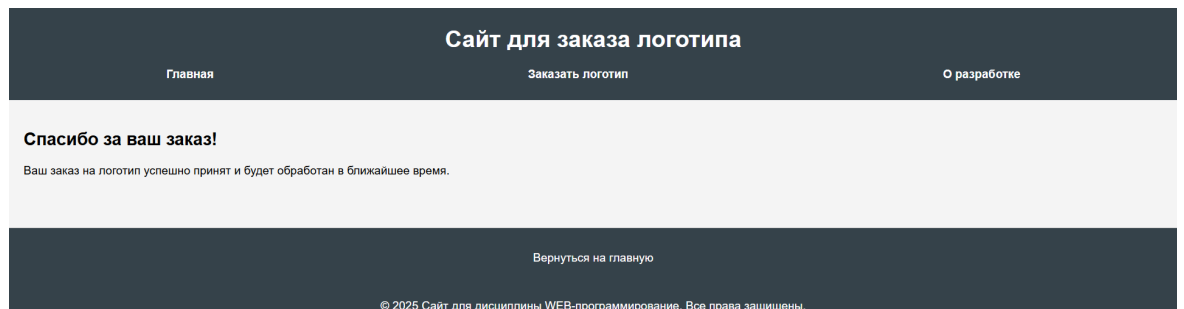


Рисунок 14 – Страница с успешным заказом

Также, при нажатии на «О разработке» в хэдере, то пользователь попадет на следующую страницу.

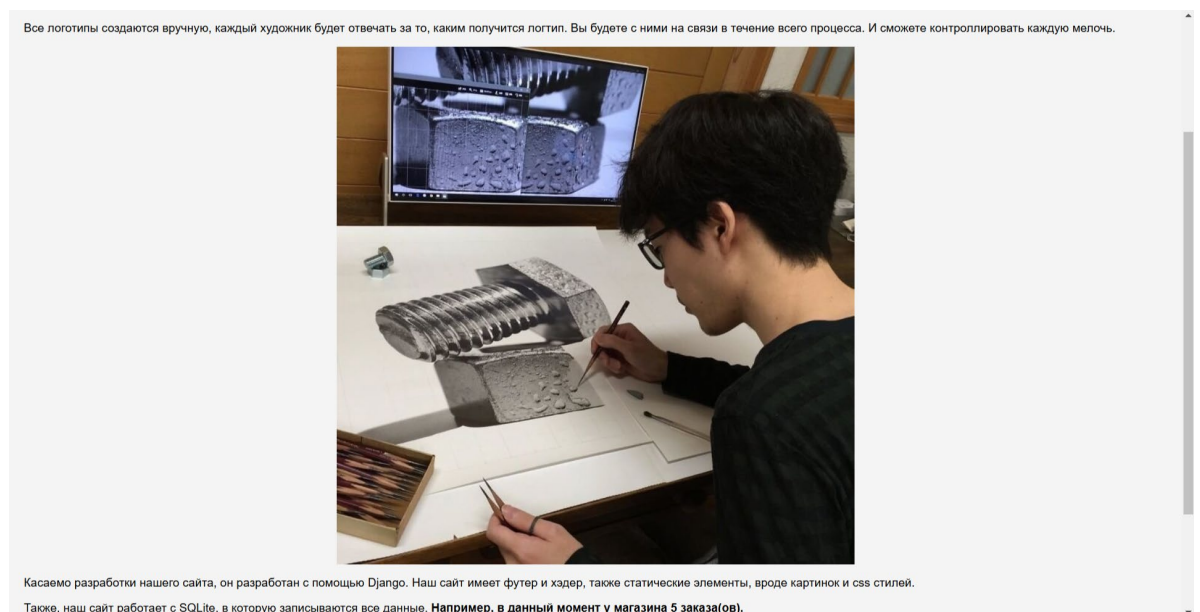


Рисунок 15 – Страница «О разработке»

Вывод

В ходе выполнения лабораторной работы цель была достигнута. Создали проект на Django с нуля, включающий три страницы, форму и футер с хэдером и используя возможности фреймворка.

В течение выполнения работы был установлен Django, написана основная архитектура, а именно футер и хэдер, написаны три страницы сайта: Главная, Заказ логотипа, О разработке. Также, была написана административная панель.

Код можно найти в репозитории:
https://github.com/NorthPole0499/WebDevelopment_2024-2025/tree/lab_7-8